Smali By bithowl: Chapter 4 DEX Fundamentals

("Every Android App Has a Blueprint… Learn to Read It")

Imagine walking into a massive library. Thousands of books. Millions of words.

But instead of shelves organised by title or author, every page is indexed, cross-referenced, and connected.

Need a word? Go to the string index.

Need a method? Check the method table.

Need a class? Look in the class definitions.

Nothing is stored randomly. Everything has an address.

That's exactly how an Android **DEX file** works. Most developers never look inside one. Most users never know it exists. But every Android reverse engineer eventually discovers that the DEX file isn't just compiled code. It's a blueprint of the entire application.

If Smali is the language Android speaks, then DEX is the book where that language is written.

Let's open it.

 # The Story (Hook)

Imagine receiving a 500-page instruction manual. You don't know the language. You don't know the symbols.

But every sentence points to another page. Every chapter references another chapter. Every object has an ID.

Instead of copying the same information repeatedly, the book simply says: "See entry #42."

That's exactly how Android keeps applications small and efficient.

Instead of writing the same class names, strings, and methods over and over again, DEX stores them once and references them using indexes.

Understanding these indexes is the first step toward understanding how Android applications are organised internally.

 # What Is a DEX File?

DEX stands for **Dalvik Executable**. It is the executable format used by Android applications. Every Java or Kotlin class is eventually compiled into one or more DEX files.

Inside an APK you'll commonly find:

classes.dex

Large applications may also include:

classes2.dex

classes3.dex

classes4.dex

Each DEX file contains the application's compiled logic in a compact, optimized format that Android Runtime (ART) can load and execute. Think of a DEX file as the application's database of code.

Everything Android needs to execute the app lives here.

 # Understanding classes.dex

If an APK is a house, then **classes.dex** is the electrical wiring hidden behind the walls.

Users never see it. Developers rarely think about it after compilation. But Android depends on it for everything.

Inside classes.dex you'll find:

- Every class
- Every method
- Every field
- Every string
- Every instruction
- Every constant
- Every reference used by the application

When APKTool converts an APK into Smali, it simply translates the information stored inside classes.dex into readable assembly code.

 # Why Does DEX Use IDs?

Imagine writing this class name thousands of times: **com.example.myapplication.LoginActivity**

That would waste space. Instead, Android stores it **once**. Every other place simply references its ID.

The same idea applies to:

- Strings
- Methods
- Fields
- Types

This makes APKs much smaller and significantly faster to process.

 # High-Level DEX Structure

At a high level, a DEX file is organised into multiple sections.

Each section stores a specific type of information.

DEX File

│

├── Header

├── String IDs

├── Type IDs

├── Proto IDs

├── Field IDs

├── Method IDs

├── Class Definitions

└── Code Items

Think of these sections as different chapters of the same book. Each chapter describes one part of the application.

DEX Header

Every DEX file begins with a **Header**. The header tells Android how to interpret the rest of the file.

It contains information such as:

- DEX version
- File size
- Checksum
- SHA-1 signature
- Offsets to each section
- Number of entries in every table

Without the header, Android wouldn't know where anything is located. It's essentially the table of contents for the entire DEX file.

String IDs

Applications contain thousands of strings.

Examples include:

"Login"

"https://api.example.com"

"Authorization"

"Access Denied"

Instead of storing these strings repeatedly, Android stores each unique string once inside the **String IDs** section. Whenever code needs a string, it references its index.

For security researchers, this section is often the quickest way to discover:

- API endpoints
- URLs
- API keys
- Secrets

- SQL queries
- Debug messages
- Feature flags

Many Android assessments begin by exploring the application's string table.

Type IDs

Everything in Java has a type.

Examples include:

java.lang.String

android.content.Intent

MainActivity

LoginFragment

The **Type IDs** section stores references to every data type used within the application.

Whenever a method creates an object or declares a parameter, Android references the corresponding Type ID instead of repeating the full class name.

Proto IDs

Methods aren't identified only by their names.

Android also needs to know:

- Return type
- Parameter types

For example: login(String username, String password)

has a different signature from: login(String username)

The **Proto IDs** section stores these method prototypes, allowing Android to distinguish methods that share the same name but accept different parameters.

Field IDs

Every class contains variables, known as fields.

For example:

private String token;

private int userId;

private boolean loggedIn;

DEX stores information about these variables inside the **Field IDs** section.

Each entry records:

- Field name

- Declaring class
- Data type

Whenever bytecode accesses a field, it references its Field ID rather than repeating all of this information.

Method IDs

Methods are the building blocks of application logic.

Examples include:

login()

logout()

fetchUser()

verifyToken()

Rather than embedding complete method details throughout the bytecode, DEX stores them once in the **Method IDs** section.

Each Method ID includes:

- Class
- Method name
- Prototype (parameters and return type)

Smali instructions reference these Method IDs whenever a method is invoked.

Class Definitions

So far we've seen:

- Strings
- Types
- Fields
- Methods

Now Android needs to know how everything fits together.

The **Class Definitions** section describes:

- Class name
- Parent class
- Interfaces
- Access modifiers
- Source file
- Class data

- Code location

This is effectively the blueprint for every class in the application.

Code Items

Everything we've explored so far describes the application's structure. **Code Items** describe its behaviour. This section contains the actual bytecode instructions executed by Android.

Inside each Code Item you'll find:

- Register count
- Method instructions
- Exception handlers
- Debug information
- Try-catch blocks

When APKTool generates a .smali file, these instructions become readable assembly code.

This is where reverse engineers spend most of their time.

 # How Everything Connects

Think of DEX like a giant relational database.

Method

|

▼

Method ID

|

▼

Prototype

|

▼

Type IDs

|

▼

String IDs

Nothing exists in isolation. Every section references another.

Understanding these relationships makes navigating Smali much easier.

 # Bug Hunter Perspective — Why DEX Structure Matters

Many beginners immediately open JADX. Experienced Android security researchers often inspect the DEX structure first.

Why?

Because the metadata itself reveals valuable information before you even read a single line of code.

For example:

String IDs

Can expose:

- Hidden API endpoints
- Hardcoded credentials
- Firebase URLs
- Encryption keys
- Debug messages
- Feature flags

Method IDs

Help identify:

- Cryptographic functions
- Authentication routines
- Root detection methods
- SSL pinning implementations
- Biometric authentication logic

Class Definitions

Reveal:

- Exported Activities
- Internal Services
- Utility classes
- Network clients
- Database managers

Code Items

Contain the actual instructions that determine how the application behaves.

For bug hunters, understanding the DEX structure means you can quickly locate the parts of an application that are most likely to contain security-sensitive logic.

Before reading Smali, you should know where to look.

 # Practical Exercise

Extract any APK using APKTool: apktool d app.apk

Then:

1. Locate classes.dex.
2. Open the APK in JADX.
3. Browse the package structure.
4. Observe how classes, methods, and fields correspond to entries within the DEX file.
5. Open a matching Smali file and compare it with the decompiled Java.

As you progress through this series, you'll begin recognising how every Smali instruction ultimately traces back to the DEX structure you've learned in this chapter.

 # Quick Recap

- DEX (Dalvik Executable) is Android's compiled executable format.
- classes.dex stores the application's compiled logic.
- DEX uses indexed tables to reduce duplication and improve efficiency.
- String IDs, Type IDs, Proto IDs, Field IDs, and Method IDs work together to describe the application's structure.
- Class Definitions organise the application's classes.
- Code Items contain the bytecode instructions executed by Android Runtime.
- Understanding the DEX structure makes reverse engineering and Smali analysis significantly easier.

 # Final Thought

Most people see an APK as a single file. Reverse engineers see something different. They see a carefully organised blueprint. Every string has an index. Every method has an identity. Every class has a definition. And every instruction has a place.

The better you understand the blueprint, the easier it becomes to understand the application itself.

Because before you can read Android's language…

You need to understand the book it's written in.

By - bithowl